

Self-Evolving Tool-Using Agents: Distilling Grounded Execution into Skill-Centric LoRA Training Data

Abstract

Tool-using language agents often fail in predictable ways: they narrate actions instead of executing them, hallucinate tool use, lose grounding across steps, and fail to accumulate reusable procedural knowledge. We present a hybrid pipeline for improving local tool-using agents without relying on large human-labeled instruction-response corpora. Our system begins with human-specified capability domains, expands them into synthetic curricula using a strong external teacher model, executes those tasks through a grounded local agent with real tools, evaluates the resulting traces, extracts reusable skills from successful episodes, and converts those grounded artifacts into LoRA training data. We then pair the adapted model with a progressively structured agent runtime, evolving from a monolithic prototype into a modular capability-aware framework with task graphs, evidence objects, validation objects, configurable web-result ranking, presentation policies, and capability-family benchmarking. The resulting system, ``8088-agent_000``, demonstrates materially improved tool-using behavior relative to the base model on system inspection, file manipulation, grounded execution, and exact task completion. Across a 100-experiment benchmark suite spanning system, filesystem, web, workspace, and factual question-answering families, the final system achieved perfect pass rates in the implemented families while maintaining low latency on deterministic tasks. The main contribution is not LoRA training alone, but a grounded execution and skill-distillation pipeline coupled to an architecture that increasingly treats agent behavior as structured planning, execution, evidence accumulation, validation, and presentation.

1. Introduction

Language-model-based agents have made rapid progress, but their performance degrades sharply when tasks require actual grounded execution. In practice, many agents describe what they would do rather than doing it, hallucinate tool calls, or fail to preserve exactness in tasks involving local system state, file operations, and multi-step workflows. These issues become especially severe in local agent systems where users expect the agent to operate on real files, real processes, real web retrieval, and real workspace state.

A second challenge is data. Standard post-training approaches often rely on expensive human supervision in the form of curated instruction-response pairs. For agents, however, the key missing competence is often not natural-language style but grounded procedural behavior. This suggests a different route: generate training data from execution itself. Instead of asking humans to label ideal outputs, we can build a pipeline that seeds capability areas, expands them into task curricula, runs those tasks through a local executor with real tools, filters and evaluates the resulting traces, extracts compact reusable skills, and uses those grounded artifacts to train a LoRA adapter.

This paper describes that approach and the resulting system. We make two linked contributions. First, we define a hybrid self-improving data pipeline combining human-seeded thematic task families, teacher-expanded synthetic curricula, grounded local execution, evaluator-gated skill extraction, and LoRA post-training. Second, we describe the architectural evolution of the runtime itself, which moved from a monolithic local-agent script into a modular capability-aware framework with structured planning, execution, evidence handling, validation,

configurable ranking, presentation, and capability-family benchmarking.

Our claim is not that synthetic data alone is sufficient, nor that the system is fully autonomous from zero supervision. Rather, the contribution is a weakly human-seeded, teacher-expanded, execution-grounded, skill-distilled pipeline for producing training material and runtime behavior that more closely resemble real agent competence.

2. Problem Statement

Tool-using agents fail in several recurring ways:

- they narrate actions instead of taking them
- they hallucinate tool usage
- they can complete one-off tasks but do not accumulate reusable procedures
- they lack compact procedural memory and explicit validation
- they often produce verbose, noisy, or poorly grounded outputs even when the underlying tools succeed

A standard language-model post-training setup does not directly solve these problems because what matters most is not only stylistic alignment but grounded procedural performance. We therefore target a different goal: train and structure the system so that execution traces, validation, and distilled skills become central artifacts.

3. Method Overview

Our system has five core components:

1. curriculum and task generation
2. grounded executor agent
3. evaluator
4. skill extractor
5. persistent storage for traces and skills

The core loop is:

- generate or sample a task
- run the local executor on the task
- capture response and execution trace
- score the episode on correctness, efficiency, and tool use
- if the quality threshold is met, extract a reusable skill
- save the raw episode and distilled skill
- later convert these into LoRA training pairs

This design separates teacher expansion from executor behavior and separates raw execution from distilled skill learning.

4. Data Generation and Curriculum Construction

The training data is hybrid rather than purely synthetic or purely human-authored.

4.1 Human-seeded capability taxonomy

We began by defining a set of target capability domains, including:

- memory retrieval
- database querying
- web search and question answering
- mathematical tool use
- shell and file manipulation
- code generation

For each domain, a human operator supplied a small number of seed tasks, typically two to three exemplars. This ensured that the curriculum remained aligned with intended use cases rather than drifting into abstract benchmark behavior.

4.2 Teacher-generated task expansion

Claude Opus 4.5 was used to expand the seed tasks into domain-specific curricula of roughly 80 to 100 tasks per domain. Crucially, the teacher model was used for **task generation only**, not for generating gold output targets. This distinction matters because our objective was to distill grounded procedural competence, not merely teacher response style.

4.3 Grounded execution traces as training substrate

The resulting prompts were executed by a local tool-using agent. Those runs produced:

- execution traces
- evaluator scores
- extracted skills

These artifacts, not the teacher outputs, formed the actual substrate for downstream LoRA adaptation. This is the central methodological point.

5. Skill Distillation and Training Corpus Construction

The local executor was not just asked to solve tasks. Its episodes were evaluated and filtered, and successful behavior was distilled into skill-like procedural representations. In effect, the system builds a training corpus from grounded traces that satisfy execution quality thresholds.

This is different from both naive self-play and standard distillation. The pipeline is:

- weakly human-seeded
- teacher-expanded
- execution-grounded
- skill-distilled
- LoRA-adapted for tool use

The resulting corpus is therefore more procedural than conversational.

6. LoRA Training Setup

We trained a LoRA adapter on top of Qwen2.5-14B-Instruct using QLoRA. The training configuration included:

- rank `r=32`
- alpha `64`
- dropout `0.05`
- batch size `2`
- gradient accumulation `8`
- maximum sequence length `4096`

The final successful training run used a Vast.ai A100 SXM4 40GB instance. It completed in approximately 2 hours 2 minutes over 1162 steps and 9292 samples, converging to a final loss of 0.714 from an initial loss near 0.90.

7. Local Deployment

After training, we deployed the adapted model locally on AmirDesktop. Rather than waiting for slow CPU quantization, we copied a pre-quantized GGUF artifact from colossus. The model was then loaded into Windows Ollama and exposed to WSL through the local endpoint. Validation confirmed correct tool-call tag behavior using the expected ``❄FUNCTION❄`` format.

This local deployment step matters because the runtime was designed to operate directly on local files, system state, and self-hosted services.

8. Observed Improvement from LoRA

The central thesis that emerged from repeated testing was:

> grounded execution + skill distillation → better tool-calling behavior than the base model

The LoRA-adapted model consistently improved behavior on:

- direct system inspection
- exact file operations
- grounded task completion
- reduced narration-only behavior
- better execution fidelity

The base model sometimes retained stronger prose style, but the adapted model was materially better at behaving like a real tool-using agent.

9. Runtime Evolution

The model improvement alone was insufficient. We discovered that many failures arose from the runtime architecture itself rather than from the model.

9.1 Monolithic prototype

The initial runtime lived in a single large script that mixed:

- routing

- parsing
- file ops
- system execution
- web handling
- presentation
- tests

This made iteration increasingly brittle.

9.2 Canonical rebuilt runtime

We established ``agent8088_000.py`` as the canonical entrypoint.

9.3 Modular framework

We then split the runtime into a package containing modules for:

- planning
- task graphs
- operation execution
- capability metadata
- tool adapters
- file-op planning
- web retrieval and ranking
- workspace state
- evidence
- validation
- presentation
- summarization
- benchmarking

9.4 Capability-aware architecture

The system now includes:

- capability registry
- capability-to-tool resolver
- task graphs
- structured operation executor
- evidence objects
- validation objects
- presentation modes
- configurable web ranking policy
- workspace introspection
- capability-family benchmarking

This was a major architectural shift from a patchwork local agent toward a reusable framework.

10. Presentation and Validation

One important lesson was that execution quality and user-facing quality are separate concerns. A system can successfully execute tasks yet still produce poor outputs if presentation is not structured. We therefore separated presentation into its own layer, with response objects and renderer policies.

Similarly, validation was elevated into a first-class typed object, allowing each operation to emit:

- whether it verified
- the verification method
- expected vs observed state
- reason for failure or mismatch

This improves both trust and debugging.

11. Evaluation Methodology

Evaluation progressed through increasingly structured suites:

- 10-query benchmark
- diverse 10
- mixed 15
- varied 20
- capability-family summaries
- 100-experiment batch

We also moved beyond prompt lists to capability-family evaluation:

- `system.inspect`
- `filesystem.read`
- `filesystem.write`
- `filesystem.append`
- `filesystem.transform\_write`
- `web.search`
- `workspace.inspect`
- `fact.answer`

This allows us to understand the system by functional competence rather than by anecdotal task selection.

12. Benchmark Results

The 100-experiment benchmark produced strong results across the implemented families.

By family:

- `system.inspect`: 20 runs, pass rate 1.0, average latency ~0.00185s
- `filesystem.list`: 10 runs, pass rate 1.0, average latency ~0.0013s
- `filesystem.read`: 5 runs, pass rate 1.0
- `filesystem.write`: 20 runs, pass rate 1.0
- `filesystem.append`: 5 runs, pass rate 1.0

- `filesystem.transform\_write`: 10 runs, pass rate 1.0
- `fact.answer`: 10 runs, pass rate 1.0, average latency ~0.585s
- `web.search`: 10 runs, pass rate 1.0, average latency ~0.569s
- `workspace.inspect`: 10 runs, pass rate 1.0, average latency ~0.0027s

These results show a clear split between deterministic structured paths and model-dependent paths. The deterministic families are nearly instantaneous and highly reliable. The model-dependent families are slower but still robust within the tested range.

13. Lessons Learned

Several lessons emerged from this process.

13.1 Grounded execution matters more than synthetic cleverness

The major gains came from real execution traces and skill distillation, not merely from prompting or synthetic answer generation.

13.2 Tool use must be architected, not merely prompted

Robust tool behavior required deterministic subsystems, structured parsing, validation, and typed file-operation semantics.

13.3 Modularity is essential

As the runtime grew, monolithic design became a liability. Encapsulation improved both velocity and reliability.

13.4 Evaluation by capability family is superior to simple prompt lists

Capability-family evaluation reveals the true operational profile of the system.

13.5 Presentation deserves its own layer

Without a dedicated presentation subsystem, otherwise successful tool runs still produce poor user-visible outputs.

13.6 Shared schemas improve generality

Capabilities, task graphs, evidence objects, validation objects, and configurable ranking policies proved to be the correct abstractions for building a more general agent framework.

14. Limitations

Despite the strong current state, several limitations remain.

- task-graph execution is still partly driven by high-level intent templates rather than completely arbitrary compositional graphs
- evidence fusion across file/system/web/workspace sources can be deeper
- capability-family scoring is still heuristic and should be enriched with more nuanced judgment metrics
- current performance reflects the implemented families, not arbitrary open-domain agent use

15. Future Work

The next major steps are:

1. make `TaskGraph.operations` drive more arbitrary compositional execution
2. deepen cross-source evidence fusion
3. improve validation semantics with richer expected/observed state capture
4. enrich benchmark scoring with latency distributions, failure buckets, and higher-resolution quality metrics
5. test held-out and more adversarial task suites beyond the current structured benchmark families

16. Conclusion

We presented a grounded execution and skill-distillation pipeline for training local tool-using agents, together with the runtime architecture that made the adapted model useful in practice. The key contribution is not simply a LoRA adapter, but the combination of:

- human-seeded capability areas
- teacher-expanded synthetic curricula
- grounded execution traces
- evaluator-gated skill distillation
- modular capability-aware runtime architecture
- capability-family evaluation

The resulting system, `8088-agent\_000`, now performs strongly across system, filesystem, web, workspace, and factual families, and has evolved from a fragile prototype into a serious local agent framework. This combination of grounded training data and structured agent architecture offers a practical path toward more reliable tool-using systems.